TriX

Normas para a conceção da linguagem e da plataforma de execução

Versão 0.1 | 22 de outubro de 2026
Proposta para adoção

A TriX é a linguagem e o conjunto de contratos de execução que pretendemos desenvolver para organizar projetos, tarefas e recursos distribuídos. Ela deve combinar uma escrita acessível, verificações fortes e integração com motores existentes, permitindo utilizar terminais e clusters de diferentes fabricantes.

Este documento estabelece a base de conceção, o comportamento esperado e os critérios de conformidade para a etapa de criação. Destina-se a quem especifica a linguagem, implementa o compilador ou interpretador, desenvolve executores, cria adaptadores e valida o produto. A arquitetura Mac e NVIDIA é a primeira referência de instalação; a linguagem deve admitir outras configurações.

O nome e a referência no feminino estão definidos. As disposições técnicas seguintes constituem uma proposta normativa organizada para adoção e evolução. A versão 0.1 não certifica uma implementação, não fixa a gramática final e não declara suporte já disponível para qualquer combinação de hardware e software.

Utilização do documento

A equipa deve adotar uma revisão, selecionar os perfis aplicáveis e ligar os requisitos identificados à implementação e à evidência de validação. As decisões ainda abertas constam do capítulo 16.

Os exemplos do capítulo 17 são ilustrativos. A fonte normativa é o texto dos requisitos, interpretado segundo o capítulo 2.

Conteúdo: 127 requisitos identificados, 24 cenários de aceitação, 19 capítulos e 24 referências técnicas.

Índice

1 Identidade e propósito 3
2 Força normativa e controlo do documento 3
3 Âmbito inicial e objetivos de conceção 4
4 Arquitetura lógica e configurações físicas 5
5 Semântica e sistema de tipos 6
6 Concorrência e interoperabilidade 7
7 Autoridade e auditoria 8
8 Contratos de tarefas e tratamento de falhas 9
9 Dados modelos e memória 10
10 Recursos prioridades e seleção de destinos 11
11 Plataformas e compatibilidade 12
12 Aceleradores gestores e fornecedores remotos 14
13 Versões distribuição e atualizações 15
14 Ferramentas documentação e experiência de utilização 17
15 Qualidade e critérios de aceitação 18
16 Etapas de criação e decisões pendentes 20
17 Exemplos ilustrativos 22
18 Glossário 23
19 Fontes técnicas 24

Os títulos do índice e as referências entre parênteses retos são ligações internas na versão editável.

1 Identidade e propósito

A grafia oficial é TriX. O nome explora a proximidade fonética com a palavra inglesa tricks e a ideia de um pequeno truque de transformação: descrever trabalho de forma clara e transformá-lo em execução adequada aos recursos disponíveis. A associação a um nome feminino e o seu apelo fazem parte da intenção criativa indicada pelo proponente.

IDN-001 Identidade. A linguagem DEVE ser designada por TriX, preservando as maiúsculas T e X. Na documentação em português, ela DEVE ser referida no feminino, por exemplo, a TriX, ela e a linguagem TriX. Identificadores técnicos podem adotar a normalização definida pelo respetivo formato.

IDN-002 Finalidade. A TriX DEVE permitir descrever, validar, planear, executar e observar trabalho distribuído, incluindo modelos de IA, ferramentas e processamento de dados. Ela DEVE tornar explícitas as condições de autorização, localização dos dados, compatibilidade e utilização de recursos.

IDN-003 Independência. A linguagem, o protocolo central e os dados de projeto NÃO DEVEM exigir um fabricante, sistema operativo, fornecedor de modelos ou gestor de cluster exclusivo. Uma extensão especializada PODE depender de um fornecedor, desde que essa dependência seja declarada.

IDN-004 Continuidade dos projetos. Mudar a infraestrutura DEVE preservar os projetos e programas cuja semântica e requisitos continuem satisfeitos. Se a nova instalação não satisfizer um requisito, a TriX DEVE explicar a incompatibilidade antes de iniciar os efeitos da tarefa.

A facilidade de utilização deve resultar da clareza dos contratos e dos diagnósticos. O apelo do nome não altera as obrigações de precisão técnica, legibilidade e comportamento previsível.

2 Força normativa e controlo do documento

As palavras em maiúsculas têm o significado definido nesta secção. As frases explicativas, exemplos e referências externas são informativos, salvo quando um requisito os incorpora expressamente.

Expressão	Significado
DEVE ou DEVEM	Requisito obrigatório para conformidade com a revisão e o perfil adotados
NÃO DEVE ou NÃO DEVEM	Comportamento proibido nesse âmbito de conformidade
RECOMENDA-SE	Orientação cuja exceção exige justificação e registo
PODE ou PODEM	Capacidade opcional, sujeita aos contratos aplicáveis
Pendente	Escolha ainda não fixada por esta revisão
Ilustrativo	Exemplo que não estabelece gramática nem garantia de implementação

NOR-001 Revisão adotada. Cada implementação DEVE declarar a revisão da especificação e os perfis que pretende cumprir. A adoção desta proposta e as alterações posteriores DEVEM ficar registadas pelo responsável do projeto. Não se presume aprovação formal apenas pela existência do documento.

NOR-002 Rastreabilidade. Os identificadores dos requisitos DEVEM permanecer estáveis. Cada entrega DEVE relacionar os requisitos aplicáveis com implementação, testes e limitações. Um requisito retirado DEVE ser marcado como substituído ou revogado; o seu identificador não deve ser reutilizado.

NOR-003 Alterações. Decisões que alterem semântica, autoridade, compatibilidade ou garantias DEVEM ser acompanhadas por um registo de decisão de arquitetura, designado ADR, com motivação, alternativas, impacto, migração e evidência de validação.

NOR-004 Perfis. A conformidade DEVE distinguir o núcleo da linguagem, a coordenação, a execução local, a execução distribuída, o acesso remoto e os clientes de controlo. Cada perfil DEVE publicar requisitos obrigatórios, capacidades opcionais e dependências de outros perfis antes de qualquer declaração de conformidade. Uma implementação NÃO DEVE excluir requisitos necessários às capacidades que efetivamente disponibiliza.

NOR-005 Exceções. Uma exceção a uma recomendação DEVE ter fundamento documentado. Uma implementação que viole um requisito obrigatório NÃO DEVE declarar conformidade integral com o perfil afetado. As decisões pendentes DEVEM ser resolvidas antes da estabilização da funcionalidade correspondente.

As normas futuras serão expressas numa especificação versionada e num conjunto de testes. As fontes técnicas do capítulo 19 fundamentam escolhas e limites; não substituem os contratos próprios da TriX nem fixam automaticamente versões mínimas do produto.

3 Âmbito inicial e objetivos de conceção

A primeira versão deve tornar executável um conjunto pequeno de conceitos: projeto, tarefa, dependência, entrada, saída, efeito, política, recurso, modelo, tentativa e resultado. O desenho pode evoluir para operações mais gerais sem exigir, de início, uma nova implementação de todos os motores de IA ou de todas as bibliotecas numéricas.

Rust oferece um precedente para propriedade e duração de recursos verificadas sem um coletor de lixo obrigatório. Python oferece uma sintaxe acessível, estruturas de alto nível e integração com extensões. Swift oferece referências úteis para concorrência estruturada, atores e transferência segura entre domínios de concorrência. A TriX deve selecionar mecanismos coerentes entre si e explicitar as garantias resultantes. [F01] [F02] [F03]

ESC-001 Primeiro subconjunto. A primeira implementação DEVE suportar contratos de tarefas, dependências, políticas e execução por adaptadores. RECOMENDA-SE validar esse subconjunto através de um SDK, planos verificáveis e executores antes de investir num compilador generalista.

ESC-002 Responsabilidades. A especificação DEVE separar linguagem e semântica, representação dos planos, sistema de execução, adaptadores e ferramentas de desenvolvimento. Uma decisão de interface gráfica NÃO DEVE alterar o significado de um programa.

ESC-003 Implementação de referência. RECOMENDA-SE um núcleo de coordenação e de execução em Rust, adaptadores Python isolados para integração com o ecossistema de IA e clientes Swift nas plataformas Apple. Essa composição DEVE permanecer uma escolha de implementação, sem obrigar todas as extensões a utilizar as mesmas linguagens.

ESC-004 Reutilização. RECOMENDA-SE utilizar motores, compiladores e mecanismos de distribuição existentes quando satisfaçam os contratos. Um novo backend de baixo nível ou kernel próprio DEVE justificar o benefício e a manutenção adicional através de evidência.

ESC-005 Limites de âmbito. A primeira versão NÃO DEVE anunciar compilação universal de bibliotecas arbitrárias, acesso uniforme a qualquer acelerador, memória global coerente ou migração instantânea de estado entre fornecedores. Capacidades desse tipo exigem especificação e validação próprias.

O sucesso inicial é executar fluxos reais com semântica estável e diagnósticos úteis. A quantidade de sintaxe inventada, de fornecedores listados ou de código reescrito não constitui, por si, critério de progresso.

4 Arquitetura lógica e configurações físicas

O conceito trinitário é aplicado a três papéis lógicos: interação, coordenação e execução. Essa organização é uma proposta de arquitetura para preservar a ideia original enquanto o número de máquinas, clusters e fornecedores pode variar.

Papel	Responsabilidade	Exemplos de instalação
Interação	Pedidos, acompanhamento e intervenção humana	Aplicação nativa, cliente web, telefone ou tablet
Coordenação	Projetos, políticas, admissão e distribuição do trabalho	Serviço único ou serviço replicado com autoridade consistente
Execução	Modelos, ferramentas e transformação de dados	Terminal, servidor, cluster próprio ou serviço externo

ARQ-001 Quantidade de recursos. A configuração DEVE admitir listas de nós, grupos, clusters e fornecedores. NÃO DEVE codificar como obrigação estrutural a existência de um cluster Mac e de um cluster NVIDIA.

ARQ-002 Papéis independentes. Um equipamento PODE exercer vários papéis autorizados. A marca ou a arquitetura da CPU NÃO DEVE determinar automaticamente autoridade, confiança ou responsabilidade sobre um projeto.

ARQ-003 Capacidades técnicas. A seleção de recursos DEVE utilizar capacidades técnicas declaradas, verificadas e observadas, conservando a distinção entre estas três fontes de informação. Recursos apenas detetados NÃO DEVEM ser apresentados como validados para executar qualquer modelo. A capacidade técnica não concede autorização de utilização.

ARQ-004 Planos de controlo e dados. A coordenação DEVE controlar autorizações e acompanhar operações sem exigir que todos os ficheiros, tensores ou resultados atravessem o coordenador. Transferências diretas DEVEM ter autorização e rastreabilidade próprias.

ARQ-005 Integração com gestores. O sistema DEVE poder operar com um executor autónomo. PODE integrar gestores como Kubernetes ou Slurm através de contratos próprios. A autoridade sobre reservas locais DEVE ser clara para impedir duas atribuições incompatíveis do mesmo recurso.

ARQ-006 Grupos de execução. Recursos usados conjuntamente numa inferência ou treino DEVEM pertencer a um grupo validado para esse modo de execução. A existência de vários destinos capazes de executar tarefas separadas NÃO DEVE provar compatibilidade para cálculo conjunto.

Kubernetes documenta mecanismos para selecionar nós com diferentes tipos de GPU. Slurm oferece recursos genéricos atribuíveis, incluindo GPUs. Estas tecnologias podem integrar a execução sem se tornarem requisitos obrigatórios da linguagem. [F15] [F16]

5 Semântica e sistema de tipos

A semântica define o significado dos programas. A especificação deve permitir distinguir um erro do programa, uma incompatibilidade do destino, uma recusa de política e uma falha durante a execução.

LNG-001 Especificação. A TriX DEVE possuir uma especificação versionada de tipos, efeitos, erros, concorrência e execução distribuída. A implementação de referência DEVE ser verificável contra essa especificação.

LNG-002 Sintaxe. A gramática aprovada DEVE definir regras lexicais, precedência, associação e resolução de nomes. Até à sua aprovação, exemplos DEVEM ser identificados como ilustrativos. O mesmo programa NÃO DEVE adquirir significado diferente apenas por mudar de fabricante ou sistema operativo.

LNG-003 Verificação. O núcleo verificado DEVE verificar tipos antes da execução. PODE inferir tipos locais. Interfaces públicas DEVEM tornar tipos e contratos consultáveis e inequívocos. Operações dinâmicas DEVEM atravessar uma fronteira identificada.

LNG-004 Dados externos. Valores provenientes de modelos, ficheiros, formatos externos ou extensões DEVEM ser validados antes da sua utilização como valores tipados. Conversões falhadas DEVEM produzir erros identificáveis.

LNG-005 Ausência e falha. A linguagem DEVE distinguir valor presente, ausência de valor e falha. Uma falha remota NÃO DEVE converter-se implicitamente numa saída vazia considerada bem-sucedida.

LNG-006 Mutabilidade. RECOMENDA-SE imutabilidade por omissão. A mutação DEVE ser explícita e obedecer a regras documentadas de propriedade, isolamento ou sincronização.

LNG-007 Recursos. A linguagem DEVE definir quem controla a duração dos recursos, quando podem ser copiados, transferidos ou partilhados e como são libertados. A escolha concreta entre empréstimos, contagem de referências e outros mecanismos permanece sujeita a decisão de conceção.

LNG-008 Memória remota. Endereços locais NÃO DEVEM atravessar uma fronteira de processo ou de rede como referências ainda válidas. A transferência DEVE utilizar dados serializáveis ou identificadores de recursos cuja validade e autorização possam ser verificadas.

LNG-009 Efeitos. Operações sobre ficheiros, rede, processos, ferramentas, modelos e recursos remotos DEVEM declarar efeitos relevantes. A especificação DEVE identificar quais as verificações estáticas e quais as verificações efetuadas no momento do acesso.

LNG-010 Capacidades de autorização. Uma declaração no programa ou numa extensão NÃO DEVE conceder autoridade por si própria. A execução de efeitos DEVE depender de capacidades de autorização atribuídas e verificadas pelo sistema.

LNG-011 Propagação de políticas. Restrições associadas aos dados DEVEM ser preservadas através de funções, coleções e tarefas. Reduzir essas restrições DEVE exigir uma operação explícita, autorizada e registada.

LNG-012 Recursos declarados. A linguagem DEVE distinguir requisitos, reservas, limites aplicáveis, objetivos e preferências. Estimativas DEVEM ser identificadas como tal, e a seleção final DEVE considerar a capacidade observada do destino.

A anotação de uma política no tipo é um mecanismo de prevenção e documentação. A sua aplicação efetiva também exige verificações no sistema de execução e isolamento nas fronteiras externas.

6 Concorrência e interoperabilidade

O programa deve expressar a relação entre tarefas e o comportamento esperado quando uma delas falha, termina ou é cancelada. A distribuição não deve apagar essas relações.

CON-001 Âmbitos concorrentes. As construções concorrentes DEVEM definir a relação entre tarefas, o encerramento do âmbito e a propagação de falhas e cancelamentos. Trabalho que sobreviva ao âmbito original DEVE exigir uma operação explícita de execução durável.

CON-002 Estado partilhado. A especificação DEVE estabelecer um modelo de partilha seguro no subconjunto verificado. PODE utilizar atores e mensagens. A passagem de mensagens DEVE respeitar tipos, autorização e regras de circulação dos dados.

CON-003 Cancelamento. A linguagem DEVE distinguir pedido, aceitação e conclusão efetiva do cancelamento. Um prazo expirado NÃO DEVE ser interpretado como prova de que a execução remota terminou.

CON-004 Resultados progressivos. Fluxos de resultados DEVEM definir ordenação, conclusão, falha e controlo da pressão de produção sobre o consumidor. Resultados parciais DEVEM ser distinguíveis dos resultados finais aceites.

CON-005 Representação intermédia. A compilação ou interpretação DEVE produzir um plano verificável com tarefas, dependências, tipos, efeitos e requisitos. O esquema desse plano DEVE ser versionado independentemente dos binários próprios de cada plataforma.

CON-006 Requisitos desconhecidos. Um executor que não compreenda uma exigência obrigatória DEVE rejeitar a tarefa antes de iniciar efeitos. Uma funcionalidade opcional só PODE ser omitida quando o contrato prevê essa alternativa.

CON-007 Outras linguagens. Integrações com Rust, Python, Swift ou outras linguagens DEVEM definir formatos, propriedade, duração dos objetos, erros, concorrência e cancelamento. RECOMENDA-SE isolamento por processo quando uma falha da extensão possa comprometer o núcleo.

CON-008 Fronteira de garantias. Código externo, bibliotecas nativas e interfaces binárias NÃO DEVEM herdar implicitamente as garantias do subconjunto verificado. Interfaces diretas DEVEM declarar a ABI ou protocolo utilizado e as combinações de versões compatíveis.

CON-009 Dependências. Módulos e extensões DEVEM possuir nomes sem ambiguidade, versões e dependências identificáveis. A instalação DEVE permitir fixar os artefactos utilizados. Importar um módulo NÃO DEVE ampliar permissões.

CON-010 Reprodutibilidade. As tarefas DEVEM poder declarar identidade exata, tolerância numérica ou critérios de qualidade como objetivo de reprodução. O sistema DEVE registar versões, configuração e sementes quando relevantes. Esses registos, isoladamente, NÃO DEVEM ser apresentados como garantia de resultados idênticos.

O plano portátil descreve o trabalho. Binários, kernels, variantes de modelos e estados de execução mantêm os seus próprios contratos de compatibilidade.

7 Autoridade e auditoria

A governação de referência organiza a execução por projetos e conserva uma cadeia de autoridade identificável. O perfil desta revisão formaliza a hierarquia de coordenação e uma linha de auditoria independente. Os mecanismos de eleição, persistência, retenção e isolamento serão detalhados nas decisões de implementação.

Papel de governação	Âmbito	Relação principal
Administrador humano	Políticas, delegação e decisão final	Supervisiona coordenação e auditoria
Orchestrator global	Coordenação transversal	Delega nos SubOrchestrators
SubOrchestrator de projeto	Trabalho e recursos autorizados do projeto	Delega nos agentes do projeto
Agentes executores	Tarefas e capacidades recebidas	Reportam pela hierarquia autorizada
Global Auditor	Supervisão independente	Reporta ao administrador humano
Project Auditors	Evidência e verificações dos projetos	Reportam ao Global Auditor

GOV-001 Autoridade humana. A TriX DEVE reconhecer a autoridade humana final sobre políticas, delegações e revogações. A execução abrangida por autorizações vigentes DEVE poder prosseguir autonomamente, sem pedidos repetidos de confirmação.

GOV-002 Contexto e hierarquia. Cada tarefa DEVE identificar projeto, autoridade emitente, âmbito e versão de política. A delegação DEVE respeitar esses limites. No perfil de referência, cada projeto DEVE ter um SubOrchestrator e um Project Auditor. O SubOrchestrator reporta ao Orchestrator global; o Project Auditor reporta ao Global Auditor, que reporta ao administrador humano. A coordenação NÃO DEVE subordinar a linha de auditoria.

GOV-003 Canais dos agentes. No perfil de governação de referência, agentes e subagentes abaixo do Orchestrator NÃO DEVEM estabelecer comunicação autónoma entre pares. Delegações e resultados DEVEM seguir as rotas autorizadas. Transferências técnicas entre executores PODEM ocorrer sem criar novos canais autónomos entre agentes.

GOV-004 Auditoria separada. As identidades e permissões dos auditores DEVEM estar separadas das dos componentes supervisionados. Credenciais operacionais NÃO DEVEM autorizar alteração ou eliminação da evidência de auditoria nem das permissões dos auditores. Alterações administrativas autorizadas DEVEM seguir um procedimento separado e produzir evidência verificável segundo o modelo de confiança publicado.

GOV-005 Políticas aplicadas. Verificações de autoridade DEVEM ocorrer nas fronteiras relevantes de acesso. Cada perfil DEVE definir a propagação das revogações, a validade de autorizações em cache e o comportamento sem ligação. Uma revogação recebida e validada DEVE impedir novos efeitos abrangidos e desencadear o tratamento do trabalho em curso. Execução offline NÃO DEVE prolongar a validade nem o âmbito previamente concedidos.

GOV-006 Coordenação replicada. Se a coordenação for replicada, a implementação DEVE impedir que coordenadores concorrentes emitam ordens incompatíveis sobre o mesmo recurso. A recuperação DEVE conservar a identidade e a autoridade dos trabalhos.

GOV-007 Conteúdo externo. Instruções presentes em documentos, respostas de modelos ou resultados de ferramentas NÃO DEVEM tornar-se políticas executáveis por si próprias. O sistema DEVE conservar a distinção entre dados recebidos e autoridade concedida.

GOV-008 Identidade dos componentes. Nós, coordenadores, clientes e fornecedores DEVEM ser autenticados nas ligações relevantes. O transporte DEVE proteger integridade e confidencialidade segundo o modelo de confiança definido. Um nome de nó ou identificador de tarefa NÃO DEVE funcionar como credencial de acesso.

GOV-009 Segredos e confiança. Credenciais DEVEM permanecer fora dos programas, planos e registos que não estejam autorizados a contê-las. A especificação DEVE publicar os seus pressupostos de confiança, incluindo a autoridade dos administradores do sistema e as limitações do isolamento utilizado.

A auditoria pode detetar desvios e produzir evidência. Ela não substitui os mecanismos que aplicam permissões no momento da execução.

8 Contratos de tarefas e tratamento de falhas

Uma tarefa é uma unidade lógica de trabalho. Uma tentativa é uma execução concreta dessa tarefa. Um efeito é uma alteração observável fora do cálculo local, como gravar um artefacto, iniciar um processo ou realizar uma chamada externa.

TAR-001 Contrato. Cada tarefa DEVE declarar entradas, saídas, efeitos permitidos, requisitos, critérios de conclusão e comportamento perante indisponibilidade. Modelos, ferramentas e políticas DEVEM ser identificados por versão ou regra explícita de resolução.

TAR-002 Identidade. Trabalho, tarefa, tentativa e efeito externo DEVEM ter identidades distinguíveis. Uma nova tentativa NÃO DEVE apagar o histórico nem criar uma nova autorização implícita.

TAR-003 Estados. O sistema DEVE distinguir pelo menos criada, validada, em fila, em execução, em cancelamento, concluída, falhada, cancelada e pendente de reconciliação. Reserva, espera ou suspensão PODEM acrescentar estados com semântica própria. Transições DEVEM ser verificadas e registadas.

TAR-004 Conclusão. Uma tarefa só DEVE ser declarada concluída depois da verificação do contrato de saída e dos efeitos conhecidos. Falta de resposta com resultado incerto DEVE produzir reconciliação quando não seja possível determinar o desfecho.

TAR-005 Repetição. A política de repetição DEVE distinguir cálculo repetível, efeitos idempotentes e efeitos que exigem reconciliação. A TriX NÃO DEVE prometer execução exatamente uma vez para efeitos arbitrários em sistemas externos.

TAR-006 Deduplicação. Efeitos repetíveis DEVEM utilizar identificação, deduplicação ou mecanismos equivalentes adequados ao destino. Quando esses mecanismos não existam, a tarefa DEVE seguir uma política explícita de verificação ou intervenção.

TAR-007 Cancelamento real. O executor DEVE comunicar se o cancelamento é cooperativo, dependente de uma etapa ou conseguido por interrupção do processo. Efeitos já realizados DEVEM permanecer registados e não podem ser considerados revertidos sem confirmação.

TAR-008 Checkpoints. Um checkpoint DEVE identificar conteúdo, versões, unidade de retoma e condições de compatibilidade. Estado lógico de uma sessão NÃO DEVE ser confundido com estado de GPU, memória de processo ou KV cache transferível.

TAR-009 Continuidade offline. Cada fluxo DEVE definir o comportamento perante perda de ligação. O modo offline NÃO DEVE ampliar permissões, exportar dados por outro canal ou criar autoridade concorrente sobre recursos partilhados.

TAR-010 Proveniência. A execução DEVE conservar a evidência necessária para identificar código, políticas, modelos, entradas, tentativas e resultados. O conteúdo registado DEVE obedecer às regras de acesso e conservação.

TAR-011 Recuperação. Cada perfil operacional DEVE definir os seus objetivos de recuperação, a possibilidade de perda de progresso e os mecanismos de reconciliação. Objetivos não medidos DEVEM ser apresentados como objetivos, sem garantia operacional indevida.

9 Dados modelos e memória

Os dados e modelos são recursos com identidade, localização e regras de utilização. O planeamento deve conhecer essas propriedades antes de escolher onde executar.

DAD-001 Classificação. Dados e artefactos DEVEM identificar projeto, acesso permitido, destinos autorizados e política de conservação. Essas propriedades DEVEM ser verificadas no planeamento e no acesso efetivo.

DAD-002 Derivados. Excertos, resumos, embeddings, transcrições, imagens extraídas e resultados intermédios DEVEM conservar as restrições aplicáveis à origem. Reclassificação ou exportação DEVE ser explícita e autorizada. A TriX NÃO DEVE alegar anonimização automática apenas por transformar os dados.

DAD-003 Identidade dos modelos. Um modelo DEVE ser identificado com a precisão necessária à tarefa: revisão dos pesos, tokenizer, formato, quantização, configuração, contexto e capacidades. Uma designação comercial isolada não deve substituir esta identidade quando a reprodução exige maior precisão.

DAD-004 Substituição. Alterar modelo, quantização, precisão ou implementação DEVE respeitar as alternativas autorizadas e os critérios de qualidade do contrato. Capacidade livre ou menor custo NÃO DEVEM justificar uma alteração silenciosa.

DAD-005 Multimodalidade. Entradas e saídas multimodais DEVEM declarar formatos e limites relevantes, incluindo dimensões, duração ou volume quando aplicável. Conversões DEVEM preservar proveniência e indicar perdas relevantes para a tarefa.

DAD-006 Memória. A admissão DEVE considerar pesos, caches, buffers, temporários e margem operacional pertinentes. A estimativa DEVE distinguir memória da CPU, memória do acelerador e memória unificada realmente partilhada.

DAD-007 Caches. A reutilização de caches DEVE respeitar modelo, configuração e autorização dos dados. O sistema DEVE impedir que uma cache constitua um caminho de acesso não autorizado entre projetos.

DAD-008 Distribuição. A localização de artefactos, transferências e réplicas DEVE ser explícita. Memória agregada entre nós NÃO DEVE ser apresentada como memória coerente utilizável por qualquer processo.

DAD-009 Consistência. A especificação DEVE definir a consistência das alterações a projetos, artefactos e metadados partilhados. Resultados produzidos sob versões diferentes DEVEM ser distinguíveis. Conflitos de escrita DEVEM ter resolução documentada.

DAD-010 Conservação e eliminação. O produto DEVE permitir aplicar políticas de conservação e eliminação. DEVE distinguir remoção lógica, revogação de acesso e garantias de eliminação física que o ambiente consiga efetivamente oferecer.

10 Recursos prioridades e seleção de destinos

A seleção deve começar pela elegibilidade. Só depois se comparam desempenho, custo, energia ou preferências de utilização.

Categoria	Significado	Exemplo
Requisito	Condição obrigatória de elegibilidade	Dados permanecem numa zona autorizada
Reserva	Compromisso contabilístico de recursos	Capacidade reservada para uma tarefa admitida
Limite	Restrição aplicável por um mecanismo identificado	Teto de memória suportado pelo ambiente
Objetivo	Resultado pretendido cuja satisfação é medida	Latência desejada

Categoria	Significado	Exemplo
Preferência	Critério de escolha entre opções elegíveis	Priorizar recursos próprios
Observação	Valor medido durante a execução	Memória e tempo efetivamente usados

REC-001 Ordem de decisão. Destinos proibidos, incompatíveis ou sem recursos admissíveis DEVEM ser excluídos antes da otimização. Prioridade, preço ou desempenho NÃO DEVEM compensar violação de um requisito obrigatório.

REC-002 Aplicação de limites. Um limite só DEVE ser anunciado como imposto quando existir um mecanismo adequado e verificado. Reserva contabilística ou observação de uso NÃO DEVE ser apresentada como isolamento físico.

REC-003 Admissão. A admissão DEVE considerar quotas, trabalho em curso e capacidade utilizável. RECOMENDA-SE proteger capacidade interativa e evitar espera indefinida de tarefas elegíveis, segundo políticas documentadas.

REC-004 Rede e localização. O planeamento DEVE considerar transferências, latência, largura de banda, disponibilidade de dados e custos de preparação relevantes. Um destino com cálculo mais rápido não deve ser escolhido sem considerar o fluxo completo.

REC-005 Capacidade partilhada. Recursos expostos por contentores, máquinas virtuais ou subsistemas sobre o mesmo equipamento DEVEM ter uma identidade física ou mecanismo equivalente de coordenação, para evitar dupla contabilização e sobre-reserva.

REC-006 Mobilidade e energia. Terminais DEVEM anunciar disponibilidade, estado de energia e limitações relevantes do sistema. A participação em tarefas DEVE respeitar as políticas do utilizador e a capacidade efetiva de continuar em segundo plano.

REC-007 Custos externos. Operações remotas DEVEM ter orçamento e política de admissão. Estimativas, custos confirmados e valores ainda incertos DEVEM ser distinguíveis. Se o fornecedor não permitir um teto estrito para um pedido em curso, essa limitação DEVE ser explícita.

REC-008 Explicabilidade. O sistema DEVE poder explicar por que escolheu ou excluiu um destino, sem revelar segredos ou dados de outros projetos. A ausência de destino elegível DEVE produzir espera, erro ou alternativa previamente autorizada.

11 Plataformas e compatibilidade

A unidade de compatibilidade inclui CPU e extensões de instruções, sistema operativo e versão, ABI, ambiente de execução, acelerador, controlador, motor, modelo e funcionalidade.

x64, x86-64, x86_64 e AMD64 designam a arquitetura x86 de 64 bits. ARM64 ou AArch64 identifica outra arquitetura. Intel Arc é uma família de GPUs. Sistemas com a mesma arquitetura de CPU podem exigir binários diferentes por causa do sistema operativo e da ABI. Os destinos Rust e as ABIs Android ilustram esta separação. [F04] [F05]

PLT-001 Identificação do destino. Cada pacote e executor DEVE declarar a combinação de plataforma pertinente. A categoria genérica Linux, Windows, ARM ou GPU não deve constituir a declaração final de compatibilidade.

PLT-002 Arquitetura observada. O inventário DEVE distinguir arquitetura física, arquitetura do processo e execução nativa, emulada ou virtualizada. Extensões opcionais da CPU DEVEM ser verificadas antes da utilização.

PLT-003 Versões mínimas. Cada publicação DEVE distinguir o mínimo para a aplicação, para a execução local e para funcionalidades ou motores específicos. O mínimo do compilador NÃO DEVE ser copiado automaticamente para o produto completo.

PLT-004 Dependências. Ambientes Python e bibliotecas nativas DEVEM ter dependências fixáveis e compatibilidade declarada por sistema, arquitetura e ABI. RECOMENDA-SE isolamento reprodutível por adaptador.

PLT-005 Linux e subsistemas. Distribuições, bibliotecas de sistema e ambientes como WSL DEVEM ter perfis próprios quando afetem execução ou aceleração. Imagens de contentor NÃO DEVEM substituir a verificação do controlador e dos recursos do anfitrião.

PLT-006 Cliente e executor. Suporte de interface, controlo remoto, CPU local, aceleração e cálculo distribuído DEVEM ser classificados separadamente. Um dispositivo PODE ser plenamente suportado como cliente sem oferecer execução local geral.

PLT-007 Dispositivos móveis. iOS, iPadOS e Android DEVEM ter perfis de ciclo de vida e disponibilidade específicos. Um cliente instalado NÃO DEVE ser anunciado como executor permanente sem mecanismos e condições que o sustentem.

PLT-008 Validação física. Compilação cruzada e testes em emulação PODEM verificar partes do núcleo. Suporte de aceleração, consumo e desempenho DEVE ser validado em hardware real representativo.

PLT-009 Extensão de plataformas. Novos destinos DEVEM poder ser acrescentados através de implementações das interfaces de plataforma e dos adaptadores aplicáveis. A gramática central NÃO DEVE criar um dialeto incompatível por fabricante.

Matriz de planeamento

A classificação seguinte orienta o desenvolvimento. Nenhuma linha constitui uma declaração de suporte implementado.

Destino	Papel previsto	Critério de entrada
macOS ARM64	Aplicação e executor Apple	Caminho completo num equipamento de referência
Linux x86_64	Servidor e cluster	Distribuição e stack de aceleração selecionadas
Windows x64	Aplicação e executor	Dependências e serviços nativos verificados
Linux ARM64	Servidor ou dispositivo especializado	Perfil próprio de dependências e aceleradores
Windows ARM64	Aplicação e executor nativos	Bibliotecas e controladores compatíveis
iOS e iPadOS ARM64	Controlo e tarefas locais	Regras de disponibilidade do sistema satisfeitas

Destino	Papel previsto	Critério de entrada
Android ARM64	Controlo e tarefas locais	Perfil de energia e tarefas em segundo plano
Navegador	Cliente de controlo amplo	Protocolo e funcionalidades do navegador
macOS Intel e outros legados	Compatibilidade por necessidade	Versões e motores ainda utilizáveis
BSD RISC V e sistemas especializados	Extensão ou investigação	Adaptador e validação próprios
x86 de 32 bits e ARM32	Cliente leve quando necessário	Benefício concreto e dependências disponíveis

Windows em ARM distingue emulação de aplicações e controladores nativos. iOS e iPadOS oferecem mecanismos condicionados de continuação de trabalho e, em dispositivos suportados, GPU em segundo plano; Android tem mecanismos próprios de tarefas e serviços. Estas diferenças devem integrar a capacidade anunciada. [F06] [F07] [F08]

12 Aceleradores gestores e fornecedores remotos

Uma integração deve representar as capacidades que consegue demonstrar. O fabricante não define, por si, o conjunto de modelos ou operações suportados.

Família	Caminho técnico a avaliar	Condição principal
Apple silicon	MLX Metal e Core ML	Motor modelo e unidades de cálculo realmente suportados
NVIDIA	CUDA motores compatíveis e NCCL	GPU controlador sistema motor e topologia qualificados
AMD	ROCm motores compatíveis e RCCL	Família concreta de hardware e matriz de versões
Intel CPU GPU e NPU	OpenVINO e componentes adequados	Perfis separados para cada classe de dispositivo
Intel Gaudi	Stack Gaudi e HCCL	Contrato próprio de software e comunicação
Qualcomm	QNN e motores compatíveis	SoC sistema operações e representação do modelo
Google Cloud TPU	Motores compatíveis como JAX	Ambiente TPU e bibliotecas correspondentes
AWS Trainium e Inferentia	Neuron	Geração motor e versões suportadas
CPU geral	Motor CPU apropriado	Instruções memória e desempenho medidos
Outros fornecedores	SDK motor ou API	Adaptador e evidência de conformidade

A matriz é um catálogo de possibilidades de integração. As fontes oficiais descrevem caminhos distintos e combinações qualificadas. MLX identifica CPU e GPU; Core ML pode selecionar unidades Apple conforme o modelo. ROCm e Gaudi mantêm matrizes próprias. QNN, TPU e Neuron exigem integrações específicas. [F09] [F10] [F11] [F12] [F13] [F14] [F17] [F18] [F19]

INT-001 Interfaces separadas. O desenho DEVE separar plataforma, descoberta de aceleradores, motor de execução, gestor de cluster e fornecedor remoto. Uma extensão PODE implementar mais de uma interface, conservando as suas responsabilidades.

INT-002 Contrato mínimo. Adaptadores que ofereçam execução de tarefas DEVEM disponibilizar identificação, descoberta de capacidades, validação de pedidos, submissão, acompanhamento, resultados e pedido de cancelamento. As restantes interfaces DEVEM publicar o contrato correspondente à sua responsabilidade. A semântica e as limitações de cada operação DEVEM ser explícitas; operações indisponíveis NÃO DEVEM ser anunciadas como suportadas.

INT-003 Capacidades opcionais. Reserva, suspensão, checkpoint, retoma, exportação de estado, prioridades e grupos de processos DEVEM ser negociados. Uma capacidade ausente NÃO DEVE ser simulada com uma garantia mais forte do que a implementação real.

INT-004 Extensões de terceiros. DEVE existir uma especificação dos contratos, SDK, exemplos e testes de conformidade. Um terceiro DEVE poder acrescentar uma integração sem modificar a semântica central da linguagem.

INT-005 Origem e confiança. Cada extensão DEVE identificar mantenedor, versão, dependências e permissões necessárias. A instalação e a atualização DEVEM verificar integridade e a política de confiança definida para essa origem.

INT-006 Validação de modelos. O reconhecimento do dispositivo ou carregamento do motor NÃO DEVE equivaler a validação de todos os modelos. Cada capacidade anunciada DEVE indicar o âmbito em que foi testada.

INT-007 Fornecedores remotos. Modelos por API DEVEM integrar os mesmos contratos de tarefas, localização de dados, credenciais, custos e auditoria. Uma chamada de ferramenta proposta por um modelo DEVE voltar ao circuito local de autorização.

INT-008 Isolamento. Adaptadores DEVEM operar com autoridade limitada ao trabalho concedido. Falhas ou reinícios DEVEM produzir estados e erros reconciliáveis. RECOMENDA-SE separar processos e ambientes de dependências quando isso reduza interferências entre motores.

INT-009 Cálculo conjunto. A divisão de uma única inferência ou treino entre fornecedores DEVE ter backend e testes específicos de operações, comunicação, precisão e recuperação. Suporte de tarefas independentes NÃO DEVE ser anunciado como tensor parallel entre quaisquer dispositivos.

NCCL, RCCL e HCCL fornecem comunicação distribuída nos seus respetivos ecossistemas. A existência de APIs semelhantes não demonstra interoperabilidade completa entre eles. PJRT oferece um precedente de interface de dispositivos por plugins que pode ser reutilizado onde adequado. [F20] [F21] [F13] [F22]

13 Versões distribuição e atualizações

O utilizador pode ver uma versão simples do produto. O inventário técnico deve conservar as versões que determinam comportamento e compatibilidade.

Elemento	Objeto versionado
Produto	Conjunto de funcionalidades distribuídas

Elemento	Objeto versionado
Linguagem	Sintaxe e significado dos programas
Plano	Esquema das tarefas e políticas serializadas
Protocolo	Mensagens e negociação entre componentes
API de extensão	Contrato implementado pelos adaptadores
Estado	Filas históricos artefactos e checkpoints persistentes
Ambiente	Motor bibliotecas Python e dependências nativas
Plataforma	Sistema ABI controladores firmware e aceleradores
Modelo	Pesos tokenizer configuração formato e quantização

VER-001 Separação. As versões da linguagem, plano, protocolo, API de extensão e estado persistente DEVEM ser identificáveis separadamente da versão comercial do produto.

VER-002 Negociação. Os componentes DEVEM negociar versões e capacidades antes da execução relevante. Funcionalidades obrigatórias desconhecidas DEVEM provocar recusa explícita. O funcionamento parcial só PODE ocorrer quando definido pelo contrato.

VER-003 Publicação. Cada publicação DEVE ter manifesto de destino, dependências, integridade, origem e combinações suportadas. Ferramentas de desenvolvimento, clientes, executores, adaptadores e modelos DEVEM poder ser distribuídos modularmente quando os contratos o permitam.

VER-004 Atualização. Atualizações DEVEM verificar compatibilidade, tratar o trabalho em curso e confirmar funcionamento antes da ativação definitiva. RECOMENDA-SE ativação gradual para alterações que afetem execução.

VER-005 Reversão. A reversão DEVE considerar o estado persistente e os checkpoints. Reinstalar um binário antigo NÃO DEVE ser apresentado como recuperação válida se esse binário não compreende os dados atualizados.

VER-006 Janela temporal. A política de versões simultaneamente suportadas DEVE ser explícita e testada. Uma regra como versão atual e anterior só PODE ser anunciada depois de definir a que componentes, versões e funcionalidades se aplica.

VER-007 Estabilidade. Alterações incompatíveis DEVEM ter identificação, migração e tratamento de programas existentes. A descontinuação de um destino DEVE indicar impacto e alternativas disponíveis, sem conversão silenciosa dos requisitos das tarefas.

VER-008 Níveis de suporte. O produto DEVE distinguir validado pelo projeto, validado pelo mantenedor, suportado com limitações, experimental, cliente remoto e não suportado. A classificação DEVE ser atribuída por funcionalidade e combinação de versões.

VER-009 Autenticidade das atualizações. Pacotes publicados DEVEM usar assinaturas ou um mecanismo de autenticidade equivalente adequado ao canal. A verificação DEVE respeitar chaves autorizadas, revogação e política de versões. Uma assinatura válida de um pacote antigo NÃO DEVE autorizar, por si só, um regresso a essa versão.

Construção reprodutível

BLD-001 Processo de construção. Cada destino suportado DEVE ter um processo de construção documentado e automatizável, com ferramentas, SDKs e requisitos identificados. Dependências especiais ou restrições que impeçam uma reconstrução independente DEVEM ser documentadas.

BLD-002 Inventário de componentes. Cada distribuição DEVE conservar um inventário dos componentes, versões, hashes e licenças utilizados. Variantes de sistema, arquitetura ou ABI NÃO DEVEM partilhar implicitamente o mesmo conjunto de dependências binárias.

BLD-003 Reprodução de artefactos. O projeto DEVE verificar a reconstrução num ambiente limpo. Quando não conseguir garantir igualdade exata dos bytes, DEVE declarar o nível de equivalência verificado e conservar a proveniência dos artefactos resultantes.

Pacotes Python e imagens para várias arquiteturas demonstram por que a distribuição precisa de distinguir destinos. O pacote não elimina a dependência da ABI, do sistema ou do anfitrião de execução. [F23] [F24]

14 Ferramentas documentação e experiência de utilização

A linguagem deve permitir compreender um programa antes de o executar e explicar o resultado depois da execução.

DEV-001 Ferramentas essenciais. O primeiro conjunto utilizável DEVE oferecer validação de programas ou planos, inspeção de capacidades, execução, acompanhamento, cancelamento e consulta de erros. RECOMENDA-SE formatação consistente e integração com editores.

DEV-002 Diagnósticos. Os erros DEVEM identificar a operação, a localização no programa quando disponível, o requisito violado e a causa conhecida. Mensagens DEVEM distinguir erro do programa, recusa de política, incompatibilidade, indisponibilidade e falha de execução.

DEV-003 Antevisão. A plataforma DEVE permitir inspecionar um plano sem executar os seus efeitos externos. A antevisão DEVE apresentar os requisitos e as incertezas relevantes, sem anunciar como reservados recursos ainda não atribuídos.

DEV-004 Documentação. Cada funcionalidade estável DEVE possuir descrição semântica, exemplos executáveis, erros relevantes e condições de suporte. Exemplos destinados ao utilizador DEVEM ser verificados nas versões indicadas.

DEV-005 Observação. O produto DEVE permitir acompanhar estados, tentativas, tempos e utilização de recursos. Telemetria e logs DEVEM minimizar dados privados e respeitar as permissões do projeto.

DEV-006 Interface comum. Clientes de plataformas diferentes DEVEM partilhar os mesmos contratos de tarefas, ações e estados. Implementações visuais PODEM ser específicas do sistema. RECOMENDA-SE acesso por teclado, mensagens legíveis e informação de estado que não dependa apenas de cor.

DEV-007 Dependências inspecionáveis. Ferramentas DEVEM permitir conhecer as versões e permissões efetivas de uma execução. RECOMENDA-SE um diagnóstico de instalação que diferencie pacote instalado, dispositivo detetado, motor utilizável e modelo validado.

DEV-008 Exemplos completos. A documentação DEVE incluir pelo menos um fluxo local, um fluxo com cluster, um fluxo com fornecedor remoto autorizado e um caso de recusa de política. Os exemplos DEVEM evidenciar o comportamento perante indisponibilidade e falha.

15 Qualidade e critérios de aceitação

A aceitação exige comportamento observável. Compilar, abrir a interface ou detetar uma GPU não basta para declarar correto um caminho de execução.

VAL-001 Evidência. Cada requisito obrigatório aplicável DEVE ter evidência de implementação e validação. A entrega DEVE incluir resultados, versões, configuração de teste e limitações conhecidas.

VAL-002 Correção. Testes DEVEM cobrir resultados corretos, recusas esperadas, isolamento, erros e recuperação. A qualidade de modelos DEVE ser avaliada segundo critérios definidos para a tarefa.

VAL-003 Desempenho. Comparações DEVEM declarar modelo, revisão, precisão, quantização, contexto, concorrência e hardware. Arranque e preparação DEVEM ser distinguidos de execução com o modelo carregado. As medições DEVEM permitir avaliar latência, capacidade, memória, energia quando disponível e custo por trabalho útil.

VAL-004 Falhas reais. Perfis distribuídos DEVEM ser avaliados com interrupções de processo, nó e rede, alterações de disponibilidade e resultados externos incertos. Os testes DEVEM verificar que não há conclusões fictícias nem repetições cegas de efeitos.

VAL-005 Extensibilidade demonstrada. Antes de anunciar independência operacional de fornecedores, o projeto DEVE demonstrar uma terceira integração real, além das duas configurações de referência, através dos contratos de extensão. Simulações PODEM ajudar ao desenvolvimento, mas não substituem essa evidência.

VAL-006 Critério de publicação. Uma publicação estável DEVE passar os cenários aplicáveis e resolver falhas que violem requisitos obrigatórios. Funcionalidades incompletas DEVEM permanecer assinaladas como experimentais ou indisponíveis.

Cenários obrigatórios por perfil

Cada cenário deve indicar preparação, estímulo, resultado esperado e evidência recolhida. A aplicação dos cenários segue a matriz de perfis exigida por NOR-004 e as capacidades efetivamente disponibilizadas. O catálogo de conformidade deve justificar exclusões, sem retirar testes necessários ao comportamento oferecido.

CA-01 Tipo externo inválido. Um adaptador devolve texto onde o contrato exige um número. O sistema rejeita o valor com erro de validação e não o propaga como dado tipado. Evidência: diagnóstico e ausência dos efeitos dependentes. Normas: LNG-003 a LNG-005, DEV-002.

CA-02 Exportação proibida. Um documento e um resumo derivado tentam seguir para um destino não autorizado. Ambos são bloqueados antes da transferência. Evidência: decisão de política, destinos recusados e nenhuma autorização ampliada. Normas: LNG-009 a LNG-011, DAD-001 e DAD-002, REC-001.

CA-03 Autoridade num documento. Uma entrada contém instruções para obter credenciais ou executar ferramentas. O conteúdo não aumenta capacidades e o efeito não autorizado é recusado. Normas: LNG-010, GOV-002, GOV-005 e GOV-007.

CA-04 Tarefa já autorizada. Um fluxo dentro da delegação vigente executa sem confirmações redundantes. Uma alteração de âmbito é bloqueada ou segue a política de autorização adicional. Durante uma partição de rede, a execução não inicia novos efeitos depois de expirar a autorização local; a revogação recebida respeita a regra do perfil. Normas: GOV-001, GOV-002 e GOV-005.

CA-05 Comunicação entre pares. Um agente tenta contornar a rota hierárquica. A mensagem é recusada. Uma transferência técnica autorizada continua possível e registada. Normas: ARQ-004 e GOV-003.

CA-06 Coordenadores concorrentes. Após uma partição de rede, duas instâncias tentam atribuir o mesmo recurso. O mecanismo de autoridade impede efeitos incompatíveis. Normas: ARQ-005, GOV-006, TAR-009 e REC-005.

CA-07 Destino incompatível. Existe capacidade livre, mas o modelo, a operação ou a versão necessária não é suportada. O sistema aguarda ou recusa segundo o contrato, sem alterar modelo ou precisão silenciosamente. Normas: IDN-004, CON-006, DAD-003 e DAD-004, REC-008.

CA-08 Requisito novo num executor antigo. O executor recebe um plano com exigência obrigatória desconhecida. A tarefa é recusada antes de produzir efeitos. Normas: CON-005 e CON-006, VER-001 e VER-002.

CA-09 Prazo e cancelamento. Uma operação remota excede o prazo. A interface distingue prazo expirado, pedido de cancelamento e confirmação de término. Efeitos já realizados permanecem identificáveis. Normas: CON-003, TAR-003, TAR-004 e TAR-007.

CA-10 Efeito externo incerto. A ligação falha depois de um destino receber uma operação com efeito. O sistema usa deduplicação ou reconciliação; não a repete cegamente. Normas: TAR-002, TAR-004 a TAR-006.

CA-11 Falha de um adaptador. Um processo de integração termina inesperadamente. O núcleo conserva identidade e estado, comunica a falha e reconcilia recursos e efeitos. Não declara sucesso sem evidência suficiente. Se a reconciliação comprovar o cumprimento do contrato, pode concluir a tarefa; enquanto houver incerteza, mantém reconciliação. Normas: CON-007 e CON-008, TAR-003, TAR-004 e TAR-011, INT-008.

CA-12 Checkpoint incompatível. A retoma é tentada com motor ou esquema incompatível. O sistema rejeita a retoma com motivo e só aceita uma alternativa prevista. Normas: TAR-008, VER-002 e VER-005.

CA-13 Memória insuficiente. O modelo cabe nos pesos anunciados, mas o contexto e os buffers excedem a capacidade admissível. A execução é recusada, adiada ou ajustada apenas segundo uma alternativa autorizada. Normas: DAD-006, REC-002 e REC-003.

CA-14 Memória em vários nós. A soma da memória de dispositivos sem grupo validado parece suficiente. O sistema não a anuncia como capacidade conjunta utilizável. Normas: ARQ-006, DAD-008 e INT-009.

CA-15 Cache entre projetos. Uma tarefa tenta reutilizar uma cache que contém dados não autorizados para o projeto. O acesso é impedido e a cache não revela conteúdo indevido. Normas: DAD-001, DAD-007 e GOV-005.

CA-16 Terminal suspenso. Um terminal perde energia, ligação ou autorização de segundo plano. Quando possível, comunica a alteração. Sem essa comunicação, a coordenação deixa de considerar confirmada a disponibilidade após o prazo declarado. Só é recuperável o progresso previamente confirmado; a perda de ligação não prova que o trabalho terminou. Normas: PLT-006 e PLT-007, REC-006, TAR-009.

CA-17 Modelo remoto e ferramentas. Uma API devolve um pedido de ferramenta. O pedido passa pela autorização local. Um limite de orçamento ou de exportação recusado impede novos pedidos abrangidos. Normas: INT-007, REC-007 e GOV-007.

CA-18 Atualização com estado. Uma atualização altera o esquema persistente. O sistema verifica migração e só anuncia reversão quando o binário anterior compreende o estado recuperado. Normas: VER-001, VER-004 e VER-005.

CA-19 Terceiro backend. Uma integração independente executa o mesmo contrato de tarefa, apresenta capacidades e erros próprios e passa os testes aplicáveis sem criar um dialeto da linguagem. Normas: PLT-009, INT-001 a INT-006 e VAL-005.

CA-20 Reprodução numérica. A mesma tarefa corre em dois motores elegíveis. Os resultados são avaliados segundo identidade, tolerância ou qualidade declarada; diferenças são reportadas sem alegar equivalência inexistente. Normas: CON-010, DAD-003 e DAD-004, VAL-002 e VAL-003.

CA-21 Instalação em arquiteturas diferentes. Pacotes de destinos distintos são identificados corretamente. Uma execução emulada não é confundida com uma nativa e a aceleração só é anunciada depois da verificação correspondente. Normas: PLT-001 a PLT-005 e PLT-008, DEV-007.

CA-22 Proteção da auditoria. Um componente operacional tenta alterar evidência ou permissões da auditoria com as suas credenciais. A operação é recusada, não modifica o estado protegido e produz evidência verificável. Os limites relativos a administradores privilegiados constam do modelo de confiança. Normas: GOV-004, GOV-009 e TAR-010.

CA-23 Construção e atualização autênticas. Uma distribuição é reconstruída num ambiente limpo e comparada segundo o nível de reprodução declarado. Um pacote alterado, de origem não autorizada ou proibido pela política de versão é recusado. Normas: BLD-001 a BLD-003, VER-003, VER-004 e VER-009.

CA-24 Nó não autenticado. Um processo apresenta o nome de um nó válido sem provar a identidade correspondente. A ligação não recebe a autoridade desse nó e não obtém dados privados. Os diagnósticos não expõem credenciais. Normas: GOV-008 e GOV-009, DEV-002 e DEV-005.

16 Etapas de criação e decisões pendentes

O desenvolvimento deve avançar por critérios de saída. As etapas não constituem um calendário nem uma estimativa de esforço.

Etapa	Entregáveis	Critério de saída
E0 Especificação	Vocabulário contratos perfis ADR e cenários	Escopo inicial definido e requisitos rastreáveis
E1 Execução mínima	SDK plano verificável executor e diagnóstico	Fluxo local completo com validação e falhas
E2 Referência distribuída	Integrações Mac e NVIDIA e controlo comum	Fluxos entre destinos com políticas e recuperação
E3 Extensibilidade	Terceiro backend e SDK de terceiros	CA-19 e conformidade aplicável demonstrados
E4 Linguagem própria	Gramática frontend ferramentas e migração	Programas e planos equivalentes nos casos de referência
E5 Ampliação	Novos sistemas aceleradores e otimizações	Perfis qualificados e evidência publicada

EVO-001 Entregas verticais. Cada etapa de implementação, a partir de E1, DEVE produzir pelo menos um fluxo utilizável de ponta a ponta, com erros e limitações observáveis. E0 DEVE produzir cenários completos especificados e rastreáveis aos requisitos. A etapa seguinte NÃO DEVE depender de anunciar como estáveis capacidades ainda não verificadas.

EVO-002 Gramática posterior. A gramática e as ferramentas próprias DEVEM preservar os contratos validados no protótipo ou documentar a migração. A criação de sintaxe não deve ocultar alterações de semântica.

EVO-003 Escolhas pendentes. Cada decisão pendente DEVE ter responsável, momento de resolução e registo de decisão. Escolhas que afetem utilizadores ou extensões DEVEM ser estabilizadas antes de promessas de compatibilidade pública.

Registo inicial de decisões

Decisão	Estado nesta revisão	Momento de resolução
Nome TriX e referência no feminino	Definido pelo proponente	Aplicação imediata
Abertura a vários fornecedores e plataformas	Objetivo definido	Obrigação arquitetural desde E0
Mac e NVIDIA como referência inicial	Configuração de trabalho	Fixar equipamentos e stacks em E0
Interação coordenação e execução como papéis lógicos	Proposta desta revisão	Adoção da arquitetura em E0
Gramática palavras-chave e extensão dos ficheiros	Pendente	Antes de estabilizar E4

Decisão	Estado nesta revisão	Momento de resolução
Sistema concreto de propriedade e valores dinâmicos	Pendente	Antes de estabilizar as garantias da linguagem
Formato do plano e transporte entre componentes	Pendente	Antes de estabilizar E1 e E2
Licença do núcleo SDK e especificação	Pendente	Antes da distribuição pública
Registo de pacotes e política de confiança	Pendente	Antes da distribuição de extensões
Janela de versões simultaneamente suportadas	Pendente	Antes da primeira publicação estável
Versões mínimas e hardware de referência	Pendente por perfil	Antes da qualificação desse perfil
Metas de desempenho recuperação e qualidade	Pendente por classe de tarefa	Antes de cada aceitação operacional
Primeiro fornecedor adicional	Pendente	Seleção para E3
Auditoria retenção e modelo de confiança	Proposta a detalhar	Antes de executar dados reais em E2

A adoção deve transformar as propostas técnicas escolhidas em decisões explícitas. Isso permite evoluir sem atribuir retroativamente ao proponente decisões de implementação que ainda não tomou.

17 Exemplos ilustrativos

Os exemplos desta secção descrevem a intenção. Não são código executável nem definem a sintaxe final da TriX.

Tarefa com seleção por capacidades

```
task resumir(documento):
    require operation = "text.generate"
    require model_profile = "resumos-validados-v1"
    require data_zone = project.allowed_zone
    limit cost = project.task_budget
    prefer low_latency
    then low_energy
    execute on eligible_resources()
    when unavailable: queue
```

O perfil de modelo identifica variantes e critérios de qualidade autorizados. A escolha entre destinos só ocorre depois de verificar política, compatibilidade e recursos. A fila é a alternativa explícita quando não existe um destino elegível.

Manifesto de um executor

```
schema: trix.capabilities/v1
node: node-example
platform:
  os: linux
  cpu_physical: x86_64
  cpu_process: x86_64
  abi: gnu
  execution: native
roles: [executor]
interfaces:
  task_plan: [1]
  adapter_api: [1]
validation:
  profile: reference-profile
  status: requires_verification
```

O manifesto exemplifica campos. Não demonstra que a máquina tem aceleração nem que um modelo foi testado. Uma implementação deve acrescentar versões, recursos, permissões e evidência pertinentes.

Escolha de infraestrutura

Um projeto pode conter vários recursos: um executor Apple, um grupo NVIDIA, um grupo AMD, um serviço Intel, um terminal e um fornecedor remoto. Cada tarefa seleciona apenas destinos elegíveis. O proprietário pode fixar um destino por política ou reprodução, sem alterar a gramática comum.

A saída de um modelo pode propor uma ação; a autorização para executar essa ação continua a pertencer ao sistema de governação.

18 Glossário

Termo	Definição utilizada
Adaptador	Componente que liga um contrato da TriX a uma implementação concreta
ABI	Convenções de compatibilidade entre componentes binários
Acelerador	Dispositivo especializado de cálculo como GPU NPU ou TPU
Ator	Unidade de estado isolado com interação por operações ou mensagens definidas
Backend	Implementação de execução de uma operação ou motor para um destino
Capacidade técnica	Funcionalidade ou recurso que um componente declara verifica ou observa poder disponibilizar
Capacidade de autorização	Direito concedido para uma operação dentro de um âmbito e de uma validade definidos
Checkpoint	Estado guardado com contrato explícito de retoma
Cluster	Conjunto de máquinas administrado para disponibilizar recursos
Efeito	Alteração observável fora do cálculo local

Termo	Definição utilizada
Executor	Componente que realiza trabalho autorizado num destino
Grupo validado	Recursos comprovadamente compatíveis para uma execução conjunta definida
Idempotência	Propriedade que permite repetir uma operação sem multiplicar o efeito pretendido
KV cache	Estado intermédio de atenção de certos modelos cuja compatibilidade depende da implementação
Plano	Representação verificável de tarefas dependências efeitos e requisitos
Perfil	Conjunto declarado de capacidades e condições de conformidade
Proveniência	Registo da origem e das versões que produziram um artefacto ou resultado
Reconciliação	Processo de determinar e regularizar o resultado de uma operação incerta
Runtime	Componentes que coordenam e realizam a execução dos planos
SDK	Ferramentas e interfaces para desenvolver integrações
Tensor parallel	Repartição de operações de um modelo por dispositivos com comunicação coordenada

19 Fontes técnicas

As fontes seguintes são informativas e foram consultadas no contexto da conceção em 22 de outubro de 2026. As matrizes de compatibilidade são evolutivas. Cada publicação da TriX deve fixar e verificar as versões concretas que utiliza. As normas próprias deste documento são propostas de projeto, não transcrições destas fontes.

F01 Rust. The Rust Programming Language, Understanding Ownership. Fundamento para propriedade, empréstimos e gestão da duração de recursos.
https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

F02 Python. The Python Tutorial. Referência para estruturas de alto nível, scripting e extensibilidade.
https://docs.python.org/3/tutorial/

F03 Swift. The Swift Programming Language, Concurrency. Referência para concorrência estruturada, atores e transferência entre domínios.
https://docs.swift.org/swift-book/LanguageGuide/Concurrency.html

F04 Rust. Platform Support. Destinos de compilação e níveis de suporte do compilador.
https://doc.rust-lang.org/rustc/platform-support.html

F05 Android. Android ABIs. Separação entre instruções, ABI e compatibilidade binária.
https://developer.android.com/ndk/guides/abis

F06 Microsoft. Add Arm support to your Windows app. Execução nativa, emulação e dependências ARM64.
https://learn.microsoft.com/en-us/windows/arm/add-arm-support

F07 Apple. Finish tasks in the background, WWDC25. Continuação de tarefas e acesso condicionado a GPU em segundo plano.
https://developer.apple.com/videos/play/wwdc2025/227/

F08 Android. Background tasks overview. Modelos de trabalho e restrições de segundo plano.
https://developer.android.com/develop/background-work/background-tasks

F09 Apple MLX. Repositório oficial. APIs, dispositivos de execução e variantes de instalação.
https://github.com/ml-explore/mlx

F10 Apple. Optimize your Core ML usage, WWDC22. Utilização de CPU, GPU e Neural Engine segundo o modelo.
https://developer.apple.com/videos/play/wwdc2022/10027/

F11 NVIDIA. CUDA Installation Guide for Linux. Arquiteturas e ambientes qualificados.
https://docs.nvidia.com/cuda/cuda-installation-guide-linux/

F12 AMD. ROCm compatibility matrix. Relações entre hardware, sistema, firmware, controladores e software.
https://rocm.docs.amd.com/en/latest/compatibility/compatibility-matrix.html

F13 Intel Gaudi. Intel Gaudi Software Suite. Componentes de execução e comunicação HCCL.
https://docs.habana.ai/en/latest/Gaudi_Overview/Intel_Gaudi_Software_Suite.html

F14 Intel OpenVINO. System Requirements. Perfis de CPU, GPU e NPU.
https://docs.openvino.ai/2026/about-openvino/release-notes-openvino/system-requirements.html

F15 Kubernetes. Schedule GPUs. Recursos de GPU e seleção de nós.
https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/

F16 Slurm. Generic Resource Scheduling. Atribuição de recursos como GPUs.
https://slurm.schedmd.com/gres.html

F17 ONNX Runtime. QNN Execution Provider. Integração com aceleradores Qualcomm e requisitos específicos.
https://onnxruntime.ai/docs/execution-providers/QNN-ExecutionProvider.html

F18 JAX. Installation. Pacotes e requisitos por plataforma e acelerador, incluindo Cloud TPU.
https://docs.jax.dev/en/latest/installation.html

F19 AWS. What is AWS Neuron. Compilação e execução em Trainium e Inferentia.
https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/what-is-neuron.html

F20 NVIDIA. NVIDIA Collective Communications Library. Comunicação entre GPUs e nós no ecossistema NVIDIA.
https://developer.nvidia.com/nccl

F21 AMD. RCCL documentation. Comunicação distribuída otimizada para GPUs AMD.
https://rocm.docs.amd.com/projects/rccl/en/latest/

F22 OpenXLA. PJRT Uniform Device API. Interface de dispositivos através de plugins.
https://openxla.org/xla/pjrt

F23 Python Packaging. Platform compatibility tags. Compatibilidade por Python, ABI e plataforma.
https://packaging.python.org/en/latest/specifications/platform-compatibility-tags/

F24 Docker. Multi platform builds. Variantes de imagens por sistema e arquitetura.
https://docs.docker.com/build/building/multi-platform/